

Proposta de Projeto — Rascunho

Disciplina: Paradigmas de Linguagens de Programação (IN1007)

Professor: Augusto Sampaio

Equipe: Amanda Melo e Igor Lima

1. Suporte ao Mecanismo de Exceções e Adiamento de Execução

Suporte ao Mecanismo de Exceções (`try / catch / finally / throw`) e Adiamento de Execução (`defer`) na **Linguagem Orientada a Objetos 1 (OO1)** via JavaCC.

2. Contexto e Motivação

Na arquitetura da **Linguagem OO1**, o fluxo de execução de métodos e mensagens é sequencial. Quando ocorre um erro durante a avaliação das instruções, o interpretador é interrompido de forma abrupta por exceções nativas da JVM, sem permitir tratamento de erro ou execução de métodos de finalização.

Enquanto trabalhos anteriores exploraram abordagens de tratamento de erros no paradigma imperativo, este projeto evolui o modelo ao estender a especificação léxica e sintática da OO1 e de seus avaliadores OO. A proposta integra o tratamento de exceções (`try / catch / finally / throw`) com o conceito de **adiamento de comandos** (`defer`) (inspirado em Go e Swift) no escopo de métodos de classes, garantindo a gestão de erros e a liberação de recursos do objeto.

3. Descrição das Funcionalidades

- **Lançamento Explícito de Erro (`throw`):** Interrompe a sequência normal do bloco e sinaliza uma exceção contendo um valor ou mensagem.
- **Captura e Tratamento (`try / catch`):** Avalia o bloco monitorado e, em caso de exceção, redireciona o fluxo para o `catch`, registrando o erro em uma variável local com escopo isolado.
- **Garantia de Finalização (`finally`):** Bloco cuja execução é garantida obrigatoriamente antes de concluir a instrução `try`, independentemente da ocorrência de exceções[cite: 2, 3].
- **Comando de Adiamento (`defer`):** Agenda um comando para ser executado obrigatoriamente no momento de saída do método atual. Múltiplos comandos `defer` declarados dentro do método são organizados em uma pilha LIFO.

4. Exemplo de Código na Linguagem Estendida

```
{
  classe ContaBancaria {
    var saldo = 100;

    metodo sacar(valor) {
      // defer: executado obrigatoriamente ao finalizar a execução do método (ordem
      LIFO)
      defer write("1. [defer] Registro de auditoria gravado no log.");
      defer write("2. [defer] Estado do objeto conta liberado.");

      try {
        if (valor > saldo) {
          throw "Saldo insuficiente para o saque";
        }
        saldo = saldo - valor;
        write("Saque efetuado com sucesso. Saldo restante: " + saldo);
      } catch (erro) {
        write("Falha ao processar saque: " + erro);
      } finally {
        // finally: executado obrigatoriamente ao final da estrutura try/catch
        write("3. [finally] Bloco de confirmacao concluido.");
      }
    }
  };

  var minhaConta = new ContaBancaria();
  minhaConta.sacar(150)
}
```

5. Especificação Léxica e Gramatical no JavaCC (Arquivo .jj)

5.1 Novas Palavras Reservadas (Tokens)

```
TOKEN : {  
    < TRY: "try" >  
  
    | < CATCH: "catch" >  
  
    | < FINALLY: "finally" >  
  
    | < THROW: "throw" >  
  
    | < DEFER: "defer" >  
  
}
```

5.2 Regras Gramaticais e Produções para a AST

```
Comando PComandoTry() : {  
    Comando tryBlock;  
    Id varErro;  
    Comando catchBlock = null;  
    Comando finallyBlock = null;  
}  
{  
    <TRY> tryBlock = PBloco()  
    <CATCH> "(" varErro = PId() ")" catchBlock = PBloco()  
    [ <FINALLY> finallyBlock = PBloco() ]  
    { return new TryCatchFinally(tryBlock, varErro, catchBlock, finallyBlock); }  
}
```

```
Comando PComandoThrow() : { Expressao e; }  
{  
    <THROW> e = PExpressao()  
    { return new Throw(e); }  
}
```

```
Comando PComandoDefer() : { Comando c; }  
{  
    <DEFER> c = PComando()  
    { return new Defer(c); }  
}
```

6. Semântica do Avaliador e Estrutura da AST

Para controlar os desvios de fluxo e o adiamento de comandos no paradigma orientado a objetos:

- **Classe `OO1Exception extends RuntimeException`:** Classe interna Java que encapsula a instância ou objeto de erro lançado pela instrução `throw` na OO1.
- **Gestão da Pilha `defer` por Método (LIFO):** O ambiente de execução dos métodos (`AmbienteExecucaoOO1`) manterá uma pilha (`Stack<Comando>`). Sempre que um nó `Defer` for avaliado, a instrução associada é empilhada. Ao finalizar a execução do método (seja por término normal ou interrupção por erro), o interpretador desempilha e executa cada comando `defer` registrado.

Nó da AST	Comportamento no Avaliador (Método executar)
Throw	Avalia a expressão do erro e dispara a exceção interna <code>OO1Exception</code> contendo o valor retornado.
TryCatchFinally	Executa o bloco <code>try</code> . Se capturar <code>OO1Exception</code> , cria um subambiente estendido contendo o parâmetro do <code>catch</code> e executa o bloco de tratamento. Executa obrigatoriamente o bloco <code>finally</code> ao final.
Defer	Armazena o comando na pilha de adiamento do método no <code>AmbienteExecucaoOO1</code> para ser executado no encerramento da execução da chamada.

7. Divisão de Tarefas e Cronograma

Fase	Atividades	Responsáveis
Fase 1: Gramática no JavaCC	Inclusão de tokens (<code>try</code> , <code>catch</code> , <code>finally</code> , <code>throw</code> , <code>defer</code>) e regras sintáticas no parser da OO1.	Amanda Melo / Igor Lima
Fase 2: Estrutura da AST	Criação das classes <code>Throw.java</code> , <code>TryCatchFinally.java</code> , <code>Defer.java</code> e <code>OO1Exception.java</code> .	Amanda Melo / Igor Lima
Fase 3: Semântica e Pilha LIFO	Implementação do controle de escopo local para o <code>catch</code> e gerenciamento da pilha de <code>defer</code> no escopo do método.	Amanda Melo / Igor Lima
Fase 4: Testes e Validação	Testes de invocações de métodos, desempilhamento do <code>defer</code> em objetos e exceções encadeadas	Amanda Melo / Igor Lima